Modeling Computation

Chapter 13

Copyright © McGraw-Hill Education. All rights reserved. No reproduction or distribution without the prior written consent of McGraw-Hill Education.

Chapter Summary

- Languages and Grammars
- Finite-State Machines with Output
- Finite-State Machines with No Output
- Language Recognition
- Turing Machines

Languages and Grammars

Section 13.1

Section Summary

- Phrase-Structure Grammars
- Types of Phrase-Structure Grammars
- Derivation Trees
- Backus-Naur Form

Introduction

- *Syntax* (form of a sentence) vs. *semantics* (meaning of a sentence)
- The sentence *the frog writes neatly* is a valid sentence according to the rules of English grammar. That is, it is syntactically correct, even though it's nonsensical (unless we are talking about a fantasy world).
- The sequence of words *swims quickly mathematics* is not a valid sentence according to the rules of English grammar.

Grammars

- The rules that specify the syntactically correct sentences of a natural language such as English are complex.
- Instead of studying natural languages, we can define *formal languages* that have well-defined rules of syntax.
- These rules of syntax are important both in linguistics (the study of natural languages) and in the study of programming languages.

An Example Grammar

1. a **sentence** is made up of a **noun phrase** followed by a **verb phrase**;
2. a **noun phrase** is made up of an **article** followed by an **adjective** followed by a **noun**, or
3. a **noun phrase** is made up of an **article** followed by a **noun**;
4. a **verb phrase** is made up of a **verb** followed by an **adverb**, or
5. a **verb phrase** is made up of a **verb**;
6. an **article** is *a*, or
7. an **article** is *the*;
8. an **adjective** is *large*, or
9. an **adjective** is *hungry*;
10. a **noun** is *rabbit*, or
11. a **noun** is *mathematician*;
12. a **verb** is *eats*, or
13. a **verb** is *hops*;
14. an **adverb** is *quickly*, or
15. an **adverb** is *wildly*.

We use these rules to form valid sentences by making a series of replacements until no more rules can be used.

An example sequence of replacements:

noun phrase verb phrase
article adjective noun verb phrase
article adjective noun verb adverb
the **adjective noun verb adverb**
the large **noun verb adverb**
the large rabbit **verb adverb**
the large rabbit hops **adverb**
the large rabbit hops quickly

Some additional valid sentences are:

a hungry mathematician eats wildly,
a large mathematician hops,
the rabbit eats quickly, etc.

But note that the following is not valid:

the quickly eats mathematician

Phrase-Structure Grammars

- A *vocabulary* (or *alphabet*) V is a finite, nonempty set of elements called *symbols*.
- A *word* (or *sentence*) over V is a string of finite length of elements of V.
- The *empty string* or *null string*, denoted by λ, is the string containing no symbols.
- The set of all words over V is denoted by V^*. A *language* over V is a subset of V^*.
- The elements of V that can not be replaced by other symbols are called *terminals*, e.g., *a*, *the*, and *rabbit* in the example grammar.
- Those that can be replaced by other symbols are called *nonterminals*, e.g., **sentence**, **noun phrase**, etc.
- The rules that specify when we can replace a string V^* with another string are called *productions* of the grammar. We denote by $z_0 \rightarrow z_1$ the production that specifies that z_0 can be replaced by z_1 within a string.

Phrase-Structure Grammars (*cont.*)

- A *phrase-structure grammar* $G = (V, T, S, P)$ consists of a vocabulary V, a subset T of V consisting of terminal symbols, a *start symbol* S from V, and a finite set of *productions* P.
- The set $N = V - T$ is the set of nonterminal symbols.
- Every production in P must contain at least one nonterminal on its left side.

  Example (Grammar 1): Let $G = (V, T, S, P)$, where $V = \{a, b, A, B, S\}$, $T = \{a, b\}$, S is the start symbol, and $P = \{S \rightarrow AbaA, A \rightarrow BB, B \rightarrow ab, AB \rightarrow b\}$.

Derivations

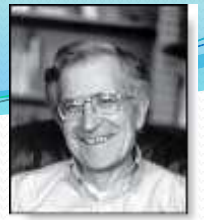
- Let $G = (V, T, S, P)$ be a phrase-structure grammar. Let $w_0 = lz_0r$ (that is the concatenation of l, z_0, and r) and $w_1 = lz_1r$ be strings over V. If $z_0 \rightarrow z_1$ is a production of G, we say that w_1 is *directly derivable* from w_0 and write $w_0 \Rightarrow w_1$.
- If $w_0, w_1, \ldots, w_n$ are strings over V such that $w_0 \Rightarrow w_1$, $w_1 \Rightarrow w_2$, ..., $w_{n-1} \Rightarrow w_n$, then we say that w_n is *derivable from* w_0 and write $w_0 \overset{*}{\Rightarrow} w_n$.
- The sequence of steps used to obtain w_n from w_0 is called a *derivation*.

Example (Grammar 1): *Aaba* is directly derivable from *ABa* because $B \rightarrow ab$ is a production and *abababa* is derivable from *ABa* because $ABa \Rightarrow Aaba \Rightarrow BBaba \Rightarrow Bababa \Rightarrow abababa$ using the productions $B \rightarrow ab$, $A \rightarrow BB$, and $B \rightarrow ab$ in both of the last two steps of the derivation.

Language Generation

- Let $G = (V, T, S, P)$ be a phrase-structure grammar. The *language generated by G,* denoted by $L(G)$, is the set of all strings or terminals that are derivable from the starting state S.
- In other words, $L(G) = \{w \in T^* \mid S \overset{*}{\Rightarrow} w\}$.
- Let G be the grammar with the vocabulary $V = \{S, A, a, b\}$, a set of terminals $T = \{a, b\}$, starting symbol S, and productions $P = \{S \rightarrow aA, S \rightarrow b, A \rightarrow aa\}$.
- $L(G) = \{b, aaa\}$, because we can begin a derivation with $S \rightarrow aA$ or with $S \rightarrow b$, and from aA we can derive aaa using $A \rightarrow aa$. There are no other possible derivations.

Types of Phrase Structure Grammars



Avram Noam Chomsky
(Born 1928)

- Phrase-structure grammars are classified by the types of allowable productions.

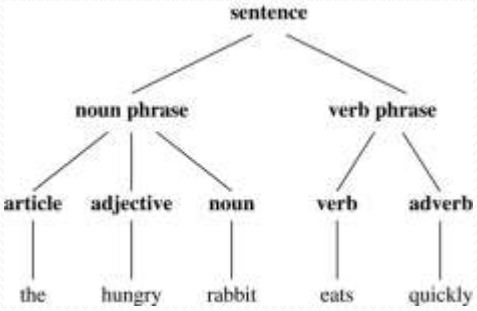
TABLE 1 Types of Grammars.

Type	*Restrictions on Productions* $w_1 \rightarrow w_2$
0	No restrictions
1	$w_1 = lAr$ and $w_2 = lwr$, where $A \in N$, $l, r, w \in (N \cup T)^*$ and $w \neq \lambda$; or $w_1 = S$ and $w_2 = \lambda$ as long as S is not on the right-hand side of another production
2	$w_1 = A$, where A is a nonterminal symbol
3	$w_1 = A$ and $w_2 = aB$ or $w_2 = a$, where $A \in N$, $B \in N$, and $a \in T$; or $w_1 = S$ and $w_2 = \lambda$

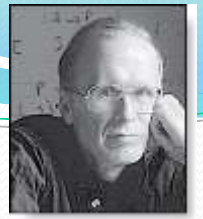
- Type 2 grammars are called *context-free grammars*. A language generated by a context-free grammar is called a *context-free language*.
- Type 3 grammars are called *context-sensitive grammars* (or a *regular grammar*). A language generated by a context-sensitive grammar is called a *context-sensitive language* (or a *regular language*).

Derivation Trees

- We can represent a derivation in the language generated by a context-free grammar by an ordered rooted tree, called a *derivation*, or *parse tree*.
  - The root of the tree represents the start symbol.
  - The internal vertices represent the nonterminal symbols that arise in the derivation.
  - The leaves represent the terminal symbols that arise.
  - If the production $A \rightarrow w$, where w is a word, arises in the derivation, the vertex that represents A has as children vertices that represent each symbol in w, in order from left to right.
- A derivation tree for the derivation of *the hungry rabbit eats quickly*, given the grammar described earlier.



Backus-Naur Form



John Backus
(1924-2007)



Peter Naur
(Born 1928)

- *Backus-Naur form* (*BNF*) is sometimes used to specify a type 2 grammar. It is often used to specify the syntactic rules of computer languages.
- The productions of a type 2 grammar have a single nonterminal symbol on their left-hand side.
- All the productions with the same nonterminal symbol on the left-hand side are combined into one statement using the symbol ::= instead of $\rightarrow$. Additionally,, all nonterminal symbols are enclosed in brackets ($\langle\rangle$), and the right-hand side of productions are spearated by bars.
- For example, the productions $A \rightarrow Aa$, $A \rightarrow a$, and $A \rightarrow AB$ are written as $\langle A \rangle ::= \langle A \rangle a \mid a \mid \langle A \rangle \langle B \rangle$.

BNF and ALGOL 60

- In the programming language ALGOL 60 an identifier consists of a string of alphanumeric characters and must begin with a letter.
- The BNF description of allowable identifiers is:

  ⟨*identifier*⟩ ::=⟨*letter*⟩ | ⟨*identifier*⟩⟨*letter*⟩ | ⟨*identifier*⟩⟨*digit*⟩

  ⟨*letter* ⟩ ::= *a* | *b* | ⋯ | *y* | *z*

  ⟨*digit*⟩ ::= 0 | 1 | ⋯ | 8 | 9
- *x*99*a* is a valid identifier since the first rule can be used to replace ⟨*identifier*⟩ by ⟨*identifier*⟩⟨*letter*⟩ , the second rule to obtain ⟨*identifier*⟩ *a*, the first rule twice to obtain ⟨*identifier*⟩⟨*digit*⟩⟨*digit*⟩ *a*, the third rule twice to obtain ⟨*identifier*⟩99*a*, the first rule to obtain ⟨*letter*⟩99*a*, and finally the second rule to obtain *x*99*a*.

Finite-State Machines with Output

Section 13.2

Section Summary

- Finite-State Machines (FSMs) with Outputs
- Types of Finite-State Machines with Outputs (*not currently included in overheads*)

Introduction

- Many kinds of machines, including computers, can be modeled using a structure called a *finite-state machine* (or *finite automaton*).
- A finite-state machine consists of a finite set of states, a designated start state, an input alphabet, and a transition function that assigns a next state to every (state, input) pair
- As we will see in Sections 13.2 – 13.4, some types of finite-state machines produce output, while for other types of finite-state machines that do not produce output some states are designated as accepting states.
- Finite-state machines are used in many diverse applications, including spell-checking programs, grammar checking, indexing, searching large bodies of text, speech recognition, XML, HTML, and network protocols

An Example of a Finite-State Machine with Output

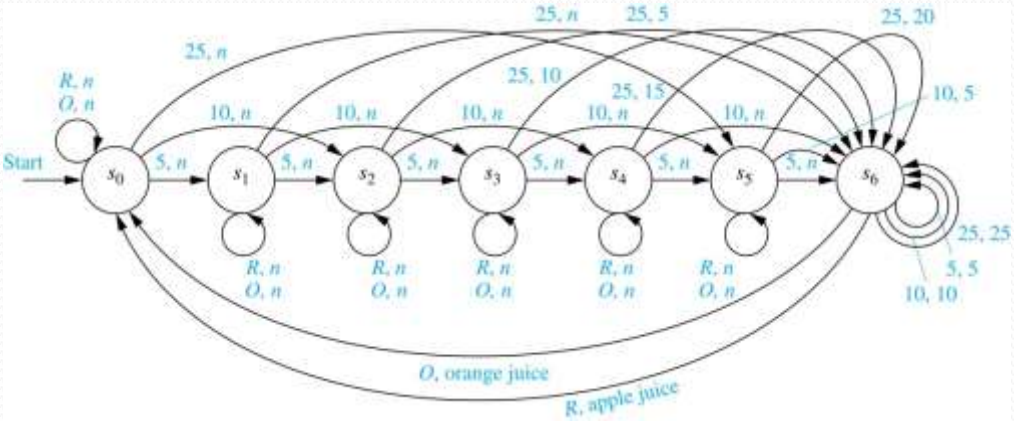
- A vending machine accepts nickels (5 cents) , dimes (10 cents) , and quarters (25 cents). When 30 cents or more has been deposited, the machine returns the amount over 30 cents. The customer can then press an orange button to receive a container of orange juice or a red button to receive a container of apple juice.
- The machine can be in any of the states s_i, $i = 0, \ldots, 6$, where s_i is the state where the machine has received $5i$ cents. The machine starts in state s_0, with 0 cents received. The possible inputs are 5 cents, 10 cents, 25 cents, the orange button (O), and the red button (R). The possible outputs are nothing (n), 5 cents, 15 cents, 20 cents, 25 cents, an orange juice, and an apple juice.

TABLE 1 State Table for a Vending Machine.

	Next State					Output				
	Input					*Input*				
State	**5**	**10**	**25**	***O***	***R***	**5**	**10**	**25**	***O***	***R***
s_0	s_1	s_2	s_5	s_0	s_0	n	n	n	n	n
s_1	s_2	s_3	s_6	s_1	s_1	n	n	n	n	n
s_2	s_3	s_4	s_6	s_2	s_2	n	n	5	n	n
s_3	s_4	s_5	s_6	s_3	s_3	n	n	10	n	n
s_4	s_5	s_6	s_6	s_4	s_4	n	n	15	n	n
s_5	s_6	s_6	s_6	s_5	s_5	n	5	20	n	n
s_6	s_6	s_6	s_6	s_0	s_0	5	10	25	OJ	AJ

An Example (*cont.*)

- We represent this vending machine using a directed graph with labeled edges, where each state is represented by a circle, edges represent transitions, and edges are labeled with the input and output for that transition.



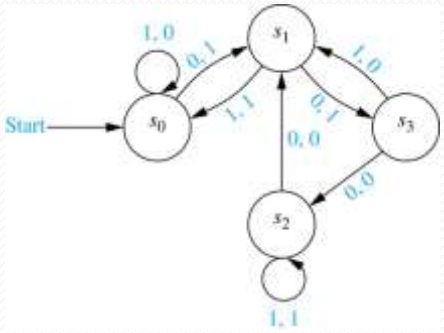
- We will trace the transitions and outputs of the vending machine when a student puts in a dime followed by a quarter, receives 5 cents back, and then pushes the orange button and receives an orange juice.
- The machine starts in state s_0.
- The first input is 10 cents, which changes the state to s_2 and gives no output.
- After the second input of 25 cents, the state changes to s_6 and gives 5 cents as output.
- The last input is the orange button, which changes the state back to s_0 and gives an orange juice as output.

FSMs with Outputs

- A *finite-state machine* $M = (S, I, O, f, g, s_0)$ consists of a finite set S of *states*, a finite *input alphabet* I, a finite *output alphabet* O, a *transition function* f that assigns to each state and input pair a new state, an *output function* g that assigns to each state and input pair an output, and an *initial state* s_0 .
- A state table is used to represent the values of the transition function f and the output function g for all (state, input).
- Alternatively, a finite-state machine can be represented by a state diagram, which is a directed graph with labeled edges. Each state is represented by a circle, and arrows labeled with the input and output pair represent the transitions.
- The state table and state diagram both represent the finite state machine with $S = \{s_0, s_1, s_2, s_3\}$, $I = \{0, 1\}$, and $O = \{0, 1\}$.

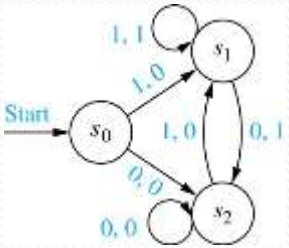
TABLE 2

	f		g	
	Input		*Input*	
State	**0**	**1**	**0**	**1**
s_0	s_1	s_0	1	0
s_1	s_3	s_0	1	1
s_2	s_1	s_2	0	1
s_3	s_2	s_1	0	0



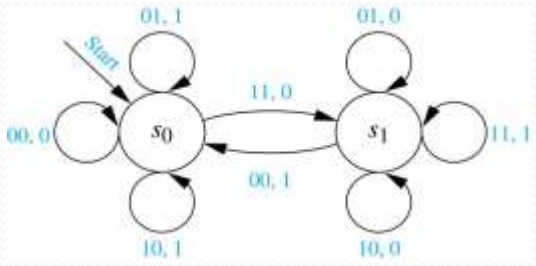
Unit-delay Machine

- An important element in many electronic devices is a *unit-delay machine*, which produces as output the input string delayed by a specified amount of time, i.e., padded with an initial string of 0s.
- How can a finite-state machine be constructed that delays an input string by one unit of time, that is, produces as output the bit string $0x_1x_2\ldots x_{k-1}$ given the input bit string $x_1x_2\ldots x_{k-1}$?
- A delay machine can be constructed that has 0 or 1 as possible inputs. The machine has the start state s_0. The transition from s_0 produces an output of 0. The machine is in state s_1 if the previous input was a 1 and it produces 1 as output for its next transition, and in state s_2 if the previous input was 0 and it produces an output of 0 for its next transition.



Addition Machine

- We will construct a finite-state machine that adds two positive integers using their binary expansions.
- Recall the conventional procedure to add $(x_n \ldots x_1 x_0)_2$ and $(y_n \ldots y_1 y_0)_2$.
  - First, the bits x_0 and y_0 are added, producing a sum bit z_0 and a carry bit c_0. Next the bits x_1 and y_1 are added together with the carry bit c_0. This gives a sum bit z_1 and a carry bit c_1.
  - The procedure continues until the nth stage, where x_n, y_n and the previous carry c_{n-1} are added to produce the sum bit z_n and the carry bit c_n, which is equal to the sum bit z_{n+1}.
- We can construct a finite state machine that uses just two states.
  - The start state s_0 is used to remember that the previous carry is 0.
  - The other state s_1 is used to remember that the previous carry is 1. (For simplicity, we assume that both x_n and y_n are 0.)
  - The inputs are pairs of bits. The transitions and the outputs are constructed from the sum of the two bits in the input and the carry represented by the state.
  - For example, when the machine is in state s_1 and receives 01 as input, the next state is s_1 and the output is 0, because the sum $0 + 1 + 1 = (10)_2$.



Finite-State Machines with No Output

Section 13.3

Section Summary

- Set of Strings
- Finite-State Automata
- Language Recognition by Finite-State Machines
- Designing Finite-State Automata
- Equivalent Finite-State Automata (*not currently included in overheads*)
- Nondeterministic Finite-State Automata

Set of Strings



Stephen Cole Kleene
(1909-1994)

- FSMs with no output, but with some states designated as *accepting states*, are specifically designed for recognizing languages.
- The *concatenation* of A and B, where A and B are subsets of V^*, denoted by AB, is the set of all strings of the form xy, where x is a string in A and y is a string in B.
- Let $A = \{0, 11\}$ and $B = \{1, 10, 110\}$. Then

  $AB = \{01, 010, 0110, 111, 110, 11110\}$ and

  $BA = \{10, 111, 100, 1011, 1100, 11011\}$
- If A is a subset of V^*, the *Kleene closure* of A, denoted by A^*, is the set consisting of arbitrarily long strings of elements of A. That is, $A^* = \bigcup_{k=0}^{\infty} A^k$.
- The Kleene closures of the sets $A = \{0\}$, $B = \{0,1\}$ and $C = \{11\}$ are $A^* = \{0^n \mid n = 0, 1, 2, ...\}$, $B^* = V^*$, and $C^* = \{1^{2n} \mid n = 0, 1, 2, ...\}$

Finite-State Automata (FSA)

- A *finite-state automaton* $M = (S, I, f, s_0, F)$ consists of a finite set S of *states*, a finite *input alphabet* I, a *transition function* f that assigns a next state to every pair of state and input (so that f: $S \times I \rightarrow S$), an *initial* or *start state* s_0, and a subset F of S consisting of *final* (or *accepting*) *states*.
- FSAs can be represented using either state tables or state diagrams, in which final states are indicated with a double circle.
- The state diagram for the FSA $M = (S, I, f, s_0, F)$, where $S = \{s_0, s_1, s_2, s_3\}$, $I = \{0, 1\}$, $F = \{s_0, s_3\}$, and the transition diagram is in Table 1, is shown here.

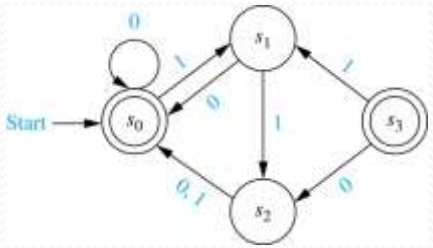


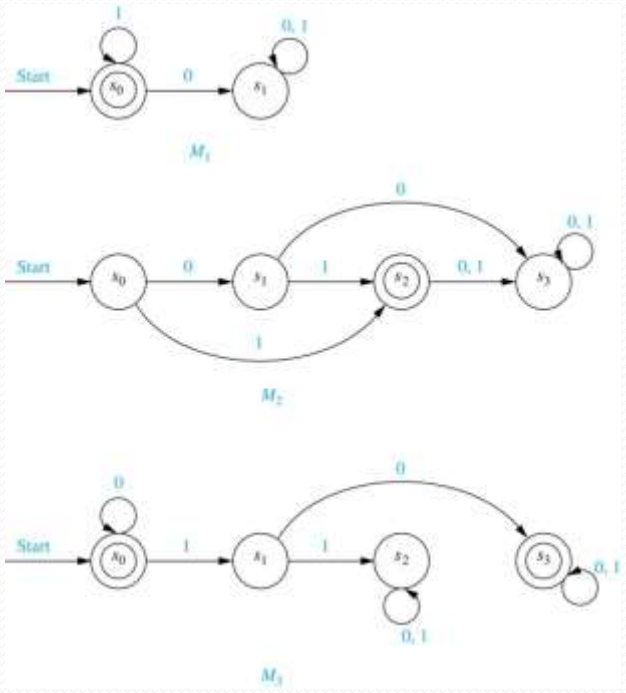
TABLE 1

	f	
	Input	
State	**0**	**1**
s_0	s_0	s_1
s_1	s_0	s_2
s_2	s_0	s_0
s_3	s_2	s_1

Language Recognition by FSAs

- A string x is said to be *recognized* (or *accepted*) by the machine $M = (S, I, f, s_0, F)$ if it takes the initial state s_0 to a final state, that is, $f(s_0, x)$. The *language recognized* (or *accepted*) by M, denoted by $L(M)$, is the set of all strings that are recognized by M. Two finite-state automata are called *equivalent* if they recognize the same language.

- The only final state of M_1 is s_0. The strings that take s_0 to itself consist of zero or more consecutive 1s. Hence, $L(M_1) = \{1^n \mid n = 0, 1, 2, ...\}$.
- The only final state of M_2 is s_2. The strings that take s_0 to s_2 are 1 and 01 . Hence , $L(M_2) = \{1, 01\}$.
- The final state of M_3 are s_0 and s_3. The strings that take s_0 to itself are λ, 0, 00, 000,... . The strings that take s_0 to s_3 are a string of zero or more consecutive 0s, followed by 10, followed by any string. Hence, $L(M_3) = \{0^n, 0^n10x \mid n = 0, 1, 2, ..., \text{ and } x \text{ is any string}\}$

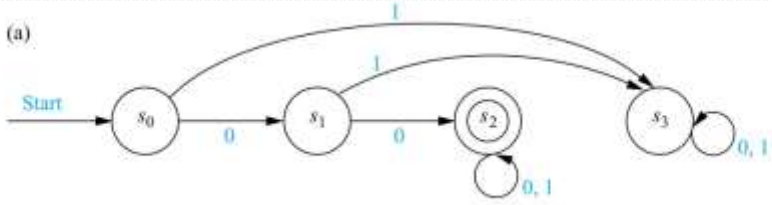


Language Recognition by FSAs (*cont.*)

Example: Construct a FSA that recognizes the set of bit strings that begin with two 0s.

Solution:

- Besides the start state s_0, we include a nonfinal state s_1; we move to s_1 from s_0 if the first bit is a 0.
- Next, we add a final state s_2, which we move to from s_1, if the second bit is a 0. We stay in this state no matter what the succeeding bits (if any) are.
- We need a nonfinal state s_3, so that we can move to it from s_0 if the first bit is a 1 and from s_1 if the second bit is a 1.

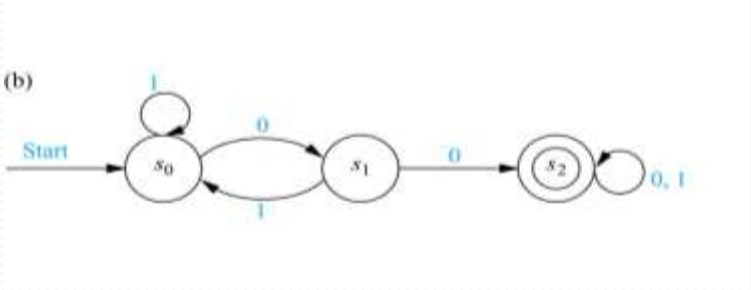


Language Recognition by FSA (*cont.*)

Example: Construct a FSA that recognizes the set of bit strings that contain two consecutive 0s.

Solution:

- Besides the start state s_0, we include a nonfinal state s_1, which tells us that the last input bit seen is a 0, but either the bit before was a 1, or this is the initial bit.
- Next, we add a final state s_2, which we move to from s_1, if the next bit after a 0 is also 0. We stay in this state no matter what the succeeding bits (if any) are.
- We return from s_0 or s_1, if a 1 follows a 0 in the string, before we come to two consecutive 0s.



NDFSA

- A nondeterministic finite-state automaton $M = (S, I, f, s_0, F)$ consists of a finite set S of *states*, a finite *input alphabet* I, a *transition function* f that assigns a set of states to every pair of state and input (so that f: $S \times I \rightarrow P(S)$), an *initial* or *start state* s_0, and a subset F of S consisting of *final* (or *accepting*) *states*.
  - We can represent a nondeterministic finite-state automaton using a state table where we give a list of possible next states for each pair of a state and an input value.
  - We construct a state diagram for a nondeterministic automaton by including an edge from each state to all possible next states, labeling edges with the input or inputs that lead to this transition.
  - We use the abbreviation NDFSA for a nondeterministic finite-state automaton and DFSA for a deterministic finite-state automata when we needed to distinguish between NDFSA and DFSA.

Example: Find the state diagram for the NDFSA with the state table shown in Table 2.
The final states are s_2 and s_3.

Solution:

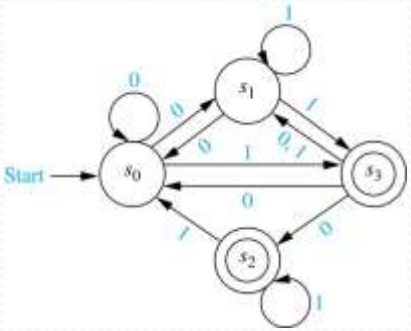


TABLE 2

State	f Input 0	f Input 1
s_0	s_0, s_1	s_3
s_1	s_0	s_1, s_3
s_2		s_0, s_2
s_3	s_0, s_1, s_2	s_1

Finding a DFSA Equivalent to a NFSA

- For every NFSA there is an equivalent DFSA. That is, if the language L is recognized by a NFSA M_0, then L is also recognized by a DFSA M_1.

  We construct the DFSA M_1 so that
  - Each state in M_1 is made up of a set of states in M_0.
  - The start symbol of M_1 is $\{s_0\}$.
  - The input set of M_1 is the same as the input set of M_0.
  - Given a state $\{s_{i_1}, s_{i_2}, \ldots, s_{i_k}\}$ of M_1, the input symbol x takes this state to the union of the sets of next states for the elements of this set, that is, the union of the sets $f(s_{i_1}, x), f(s_{i_2}, x), \ldots, f(s_{i_k}, x)$.
  - We continue in this way to construct the states of M_1 from those of M_0.
  - The final states of M_1 are those sets that contain a final state of M_0.
- To see that M_0 and M_1 are equivalent, first suppose that an input string is recognized by M_0. This means that one of the states that can be reached from s_0 is a final state. So, in M_1 this input string leads from $\{s_0\}$ to a set of states of M_0 that contains the final state. Since this is a final state of M_1, this string is also recognized by M_1.
- Conversely, a string that is not recognized by M_0 does not lead to any final states in M_0. Consequently, this input string does not lead from $\{s_0\}$ to a final state of M_1.

Finding an Equivalent DFSA (*cont.*)

Example: Find a DFSA that recognizes the same language as the NFSA:

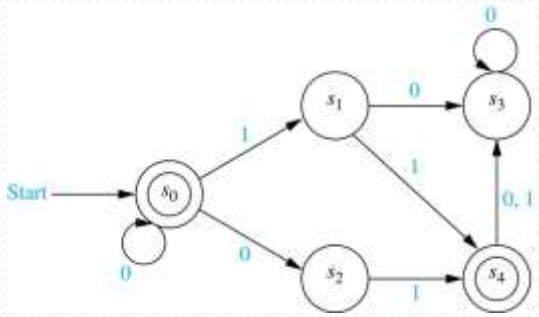
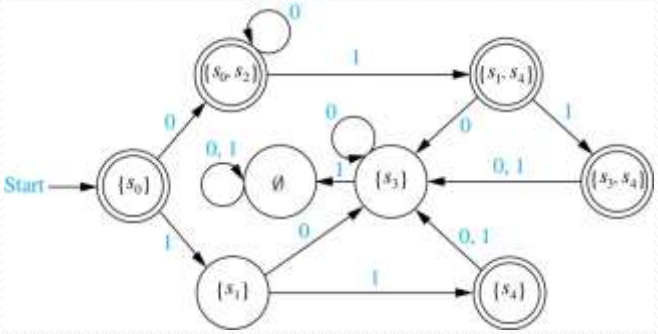


TABLE 3

State	f Input 0	1
s_0	s_0, s_2	s_1
s_1	s_3	s_4
s_2		s_4
s_3	s_3	
s_4	s_3	s_3

Solution: Following the steps of the procedure described on the previous slide, we obtain the DFSA shown here.



Language Recognition

Section 13.4

Section Summary

- Regular Expressions
- Kleene's Theorem
- Regular Sets and Regular Grammars
- More Powerful Types of Machines

Regular Expressions

- The *regular expressions* over a set I are defined recursively by:

  the symbol $\varnothing$ is a regular expression;
  the symbol λ is a regular expression;
  the symbol x is a regular expression whenever $x \in I$;
  the symbols $(\mathbf{AB})$, $(\mathbf{A} \cup \mathbf{B})$, and $\mathbf{A}^*$ are regular expressions whenever $\mathbf{A}$ and $\mathbf{B}$ are regular expressions.

- Each regular expression represents a set specified by these rules:

  $\varnothing$ represents the empty set, that is, the set with no strings;
  $\boldsymbol{\lambda}$ represents the set $\{\lambda\}$, which is the set containing the empty string;
  $\boldsymbol{x}$ represents the set $\{x\}$ containing the string with one symbol x;
  $(\mathbf{AB})$ represents the concatenation of the sets represented by $\mathbf{A}$ and by $\mathbf{B}$;
  $(\mathbf{A} \cup \mathbf{B})$ represents the union of the sets represented by $\mathbf{A}$ and by $\mathbf{B}$;
  $\mathbf{A}^*$ represents the Kleene closure of the set represented by $\mathbf{A}$.

- Sets represented by regular expressions are called *regular sets*.

Regular Expressions (*cont.*)

Example: What are the strings in the regular sets specified by the regular expressions **10***, (**10**)*, **0** ∪ **01**, **0**(**0** ∪ **1**)*, and (**0*1**)*?

Solution:

TABLE 1

Expression	*Strings*
10*	a 1 followed by any number of 0s (including no zeros)
(10)*	any number of copies of 10 (including the null string)
0 ∪ **01**	the string 0 or the string 01
0(0 ∪ **1)***	any string beginning with 0
(0*1)*	any string not ending with 0

Finite-State Automata, Regular Sets, and Regular Grammars

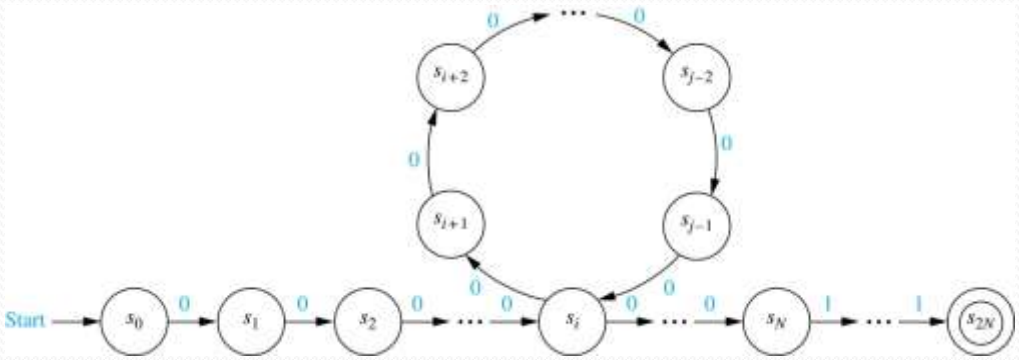
- In 1956 Kleene established the connection between regular sets and sets recognized by a FSA.
  - He showed that a set is regular if and only if it is recognized by a FSA. This result is known as *Kleene's theorem*.
  - See the text for the lengthy proof of this theorem.
- There is a close connection between regular grammars and regular sets.
  - Specifically, a set is generated by a regular grammar if and only if it is a regular set.
  - See the text for a proof.
  - We will give an example of a set that is not regular later in this section by finding a set that is not recognized by an FSA.

A Set Not Recognized by a FSA

- The set $\{0^n1^n \mid n = 0, 1, 2, \ldots\}$ of all strings consisting of a block of 0s followed by a block of an equal number of 1s is not regular.
- To show that this set is not regular, suppose that this set was regular. Then there would be a NDFSA $M = (S, I, f, s_0, F)$ recognizing it.
- Let N be the number of states in this machine, i.e., $N = |S|$.
- M must recognize 0^N1^N since it is made up of a block of 0s followed by a block of an equal number of 1s.
- Let $s_0, s_1, s_2, \ldots, s_{2N}$ be the sequence of states obtained starting at s_0 and using the symbols of 0^N1^N as input. So, $s_1 = f(s_0, 0)$, $s_2 = f(s_1, 0)$, ..., $s_N = f(s_{N-1}, 0)$, $s_{N+1} = f(s_N, 1)$, ..., $s_{2N} = f(s_{2N-1}, 1)$, and s_{2N} is a final state.
- Because there are only N states, by the pigeonhole principle at least two of the first $N + 1$ states $s_0, s_2, \ldots, s_N$ must be the same.
- Suppose that s_i and s_j are identical states with $0 \leq i < j \leq N$. This means that $f(s_i, 0^t) = s_j$, where $t = j - i$. Hence, there is a loop leading from s_i back to itself, using 0 a total of t times.

continued

A Set Not Recognized by a FSA (*cont.*)



- Now consider the input string $0^N0^t1^N = 0^{N+t}1^N$. The string is not of the correct form and so, it is not recognized by M.
- Consequently, $f(s_0, 0^{N+t}1^N)$ can not be a final state.
- However, when we use the string $0^{N+t}1^N$ as input, we end up in the same state as before, namely, s_{2N}. The reason is that we go through the loop one more time.
- This contradiction shows that $\{0^n1^n \mid n = 0, 1, 2, \ldots\}$ is not regular.

More Powerful Types of Machines

- The main limitation of finite-state automata is their finite amount of memory. This has led to the development of more powerful types of machines.
  - *Pushdown Automaton* (*PDA*): includes a stack, which provides unlimited memory. We can use a PDA to recognize $\{0^n1^n \mid n = 0, 1, 2, \ldots\}$, but no PDA recognizes the set $\{0^n1^n2^n \mid n = 0, 1, 2, \ldots\}$.
  - *Linear Bounded Automaton* (*LBA*): More powerful than pushdown automata. We can use a LBA to recognize $\{0^n1^n2^n \mid n = 0, 1, 2, \ldots\}$, but there are languages generated by phrase-structure grammars that cannot be recognized by a LBA.
  - *Turing Machine* (*TM*): Yet more powerful machines (to be studied in the next section) which can recognize all languages generated by phrase-structure grammars.

Turing Machines

Section 13.5

Section Summary

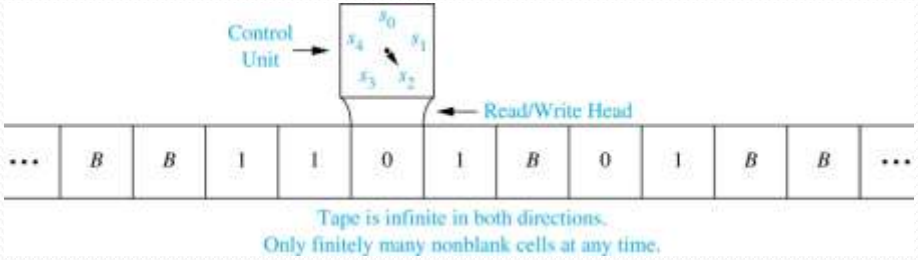
- Definition of Turing Machines
- Using Turing Machines to Recognize Sets
- Computing Functions with Turing Machines
- Different Types of Turing Machines (*not currently included in overheads*)
- The Church-Turing Thesis
- Computational Complexity, Computability, and Decidability

Alan Mathison Turing
(1912-1954)



Introduction

- Informally, a Turing machine consists of a *control unit*, which at any step is in one of finitely many different states, together with a *tape*, infinite in both directions, which is divided into *cells*.
- Turing machines have read and write capabilities on the tape as the control unit moves back and forth along this tape, changing states depending on the tape symbol read.
- Turing machines are more powerful than finite-state machines because they include additional memory capability.
- Turing machines are the most general models of computation; essentially they can do whatever a computer can do.



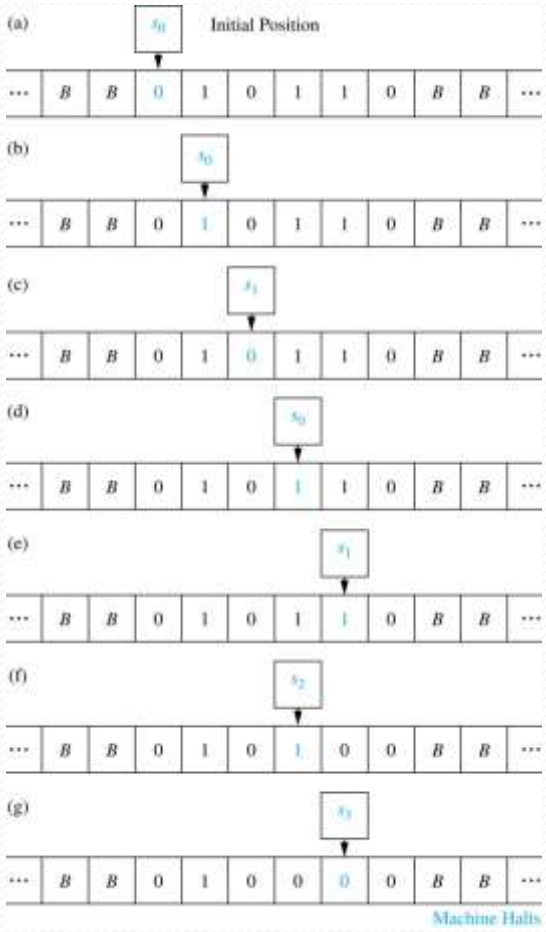
Definition of Turing Machines (TM)

- A *Turing machine* $T = (S, I, f, s_0)$ consists of
  - a finite set S of states,
  - an alphabet I containing the blank symbol B,
  - a partial function f from $S \times I$ to $S \times I \times \{R,L\}$, and
  - a starting state s_0.
- For some (state, symbol) pairs the partial function f may be undefined, but for a pair for which it is defined, there is a unique (state, symbol, direction) triple associated to this pair.
- The five-tuples corresponding to the partial function in the definition of a TM are called the *transition rules* of the machine.
- At each step, the control unit reads the current tape symbol x. If the control unit is in state s and if the partial function f is defined for the pair (s, x) with $f(s, x) = (s', x', d)$, the control unit:
  - enters the state s',
  - writes the symbol x' in the current cell, erasing x, and
  - moves right one cell if $d = R$ or moves left one cell if $d = L$.
- This step is written as the five-tuple (s, x, s', x', d). Turing machines are defined by specifying a set of such five-tuples. If the partial function f is undefined for the pair (s, x) then T will *halt*.
- At the beginning of its operation a TM is assumed to be in the initial state s_0 and to be positioned over the leftmost nonblank symbol on the tape. This is the *initial position* of the machine.

A TM in Operation

Example: What is the final tape when the TM T defined by the seven five-tuples $(s_0, 0, s_0, 0, R)$, $(s_0, 1, s_1, 1, R)$, (s_0, B, s_3, B, R), $(s_1, 0, s_0, 0, R)$, $(s_1, 1, s_2, 0, L)$, (s_1, B, s_3, B, R), and $(s_2, 1, s_3, 0, R)$ is run on the tape shown here in (a)?

Solution: The transitions of this TM are shown to the right. The final tape is shown in (g).



Using TM to Recognize Sets

- Let V be a subset of an alphabet I. A TM $T = (S, I, f, s_0)$ *recognizes* a string x in V^* if and only if T, starting in the initial position when x is written on the tape, halts in a final state.
- T is said to *recognize* a subset A of V^* if x is recognized by T if and only if x belongs to A.
- Note that to recognize a subset A of V^* we can use symbols not in V. This means that the input alphabet I may include symbols not in V. We will see that these extra symbols are used as markers.
- A TM operating on a tape containing the symbols of a string x in consecutive cells, does not recognize x if it does not halt or halts in a state that is not final.

Using TMs to Recognize Sets (*cont.*)

- In Section 13.4 we showed that there is no DFA that recognizes the set the set $\{0^n1^n \mid n \geq 1\}$. We will now construct a TM that recongizes this set.
  - We use an auxiliary tape symbol M as a marker, and specify that $V = \{0, 1\}$ and $I = \{0, 1, M\}$.
  - Our TM has one final state, s_6. The TM successively replaces a 0 at the leftmost position of the string with an M and a 1 at the rightmost position of the string with an M, sweeping back and forth, terminating in a final state if and only if the string consists of a block of 0s followed by a block of the same number of 1s.
  - The five-tuples are $(s_0, 0, s_1, M, R)$, $(s_1, 0, s_1, 0, R)$, $(s_1, 1, s_1, 1, R)$, (s_1, M, s_2, M, L), (s_1, B, s_2, B, L), $(s_2, 1, s_3, M, L)$, $(s_3, 1, s_3, 1, L)$, $(s_3, 0, s_4, 0, L)$, (s_3, M, s_5, M, R), $(s_4, 0, s_4, 0, L)$, (s_4, M, s_0, M, R), and (s_5, M, s_6, M, R).
- For example, the string 000111 would successively become $M00111$, $M0011M$, $MM011M$, $MM01MM$, $MMM1MM$, $MMMMMM$ as the machine operates until it halts.

Computing Functions with TMs

- A Turing machine can be used to compute the values of a partial function.
- Suppose that the TM T, when given the string x as input, halts with the string y on its tape. We can then define $T(x) = y$.
- To consider a TM as a computer of functions from the set of k-tuples of nonnegative integers to the set of nonnegative integers, we use the *unary representation* of integers.
- A nonnegative integer n is represented by a string of $n + 1$ 1s. So, 0 is represented by 1, 5 by 111111, etc.
- To represent an input that is a k-tuple of integers, we represent each integer in the k-tuple separately and separate these representations using asterisks. For example, (2,0,1,3) is represented by 111*1*11*1111.
- Constructing a Turing machine that computes a particular function can be extremely complicated. Fortunately, the ability of Turing machines to compute functions is of theoretical, rather than practical, interest.

Computing Functions with TMs (*cont.*)

- To construct a TM T that computes the function $f(n_1,n_2) = n_1 + n_2$, we first represent the pair (n_1,n_2) by a string of $n_1 + 1$ 1s followed by an asterisk, followed by $n_2 + 1$ 1s.
- The machine starts at the leftmost 1 of the input string, and proceeds to erase this 1.
- If the next character is an asterisk, $n_1 = 0$. In this case, it replaces the asterisk with a blank and halts.
- Otherwise, it erases the next 1, and then passes over the remaining 1s, until it comes to the asterisk.
- The asterisk is then replaced by a 1.
- The five-tuples defining this Turing machine are: $(s_0, 1, s_1, B, R)$, $(s_1, *, s_3, B, R)$, $(s_1, 1, s_2, B, R)$, $(s_2, 1, s_2, 1, R)$, $(s_2, *, s_3, 1, R)$.

The Church-Turing Thesis

- The *Church-Turing Thesis* says that given any problem that can be solved with an effective algorithm, there is a TM that can solve this problem.
- It is called a *thesis* rather than a theorem because the concept of solvability by an effective algorithm is informal and imprecise, as opposed to the concept of solvability by a TM, which is formal and precise.

Decidability and Complexity

- A *decision problem* asks whether statements from a particular class of statements are true. Decision problems are also known as *yes-or-no problems*.
  - Consider the question for a particular integer n, "Is n prime?" The answer is "yes" or "no."
  - The *halting problem* is the decision problem that asks whether a Turing machine T eventually halts when given an input string x.
- When there is an effective algorithm that decides whether instances of a decision problem are true, we say that this problem is *solvable* or *decidable*.
  - In Section 3.5, an algorithm was given for determining whether a positive integer n is prime by checking whether it is divisible by primes not exceeding its square root.
- However, if no effective algorithm exists for solving a problem, then we say the problem is *unsolvable* or *undecidable*.
  - The halting problem is an unsolvable decision problem (proved in Section 3.1). That is, no TM exists that, when given an encoding of a TM T and its input string x as input, can determine whether T eventually halts when started with x written on its tape.
- A function that can be computed by a TM is called *computable* and a function that cannot be computed by a TM is called *uncomputable*.
  - The *busy beaver function*, which when given a positive integer n gives the maximum number of 1s that a TM with n states and alphabet $\{1,B\}$ may print on an initially blank tape is uncomputable (see Exercise 35 in Section 13.5).

The Classes *P* and *NP*

- In a *nondeterministic Turing machine* (*NDTM*), the restriction that no two transition rules begin with the same pair (*s*, *x*) is eliminated.
- Hence, there may be more than one transition rule beginning with each (state, tape symbol) pair, so that there may be a choice as to which rule to use at each step.
- A NDTM *T* recognizes a string *x* if and only if there exists some sequence of transitions of *T* that ends in a final state when the machine starts in the initial position with *x* written on the tape.

The Classes *P* and *NP* (*cont.*)

- A decision problem is in *P*, the *class of polynomial-time problems*, if it can be solved by a deterministic Turing machine in polynomial time in terms of the size of its input.
- That is, a decision problem is in *P* if there is a deterministic Turing machine *T* that solves the decision problem and a polynomial $p(n)$ such that for all integers n, *T* halts in a final state after no more than $p(n)$ transitions whenever the input to *T* is a string of length n.
- A decision problem is in *NP*, the *class of nondeterministic polynomial-time problems*, if it can be solved by a nondeterministic Turing machine in polynomial time in terms of the size of its input.
- That is, a decision problem is in *NP* if there is a nondeterministic Turing machine *T* that solves the problem and a polynomial $p(n)$ such that for all integers n, *T* halts for every choice of transitions after no more than $p(n)$ transitions whenever the input to *T* is a string of length n.
- Problems in *P* are called *tractable*, whereas problems not in *P* are called *intractable*.

The Classes *P* and *NP* (*cont.*)

- For a problem to be in *NP*, it is necessary only that there be a NDTM that when given a true statement from the set of statements addressed by the problem, can verify its truth in polynomial time by making the correct guess at each step.
  - The problem of determining whether a given graph has a Hamilton circuit is an *NP* problem, because a NDTM can easily verify that a simple circuit in a graph passes through each vertex exactly once.
  - It can do this by making a series of correct guesses corresponding to successively adding edges to form the circuit.

Wrapping Everything up with a Millenium Problem

- Because every DTM can also be a considered to be a NDTM, where each (state, tape symbol) pair occurs in exactly one transition rule, P $\subseteq$ NP.
- The most famous open question in theoretical CS, and one of the millennium problems with a $1,000,000 prize, is whether every problem in *NP* is also in *P*, that is, whether *P* = *NP*.
- There is an important class of problems, known as *NP-complete* problems, where a problem is in this class if it is the class *NP* and if this problem was also in the class *P*, then every problem in *NP* must also be in *P*.
- That is, a problem is *NP*-complete if the existence of a polynomial-time algorithm for solving it implies the existence of a polynomial-time algorithm for every problem in *NP*.
- We have studied several problems that can be shown to be *NP*-complete in this text, including determining whether a simple graph has a Hamilton circuit and determining whether a proposition in *n* variables is a tautology.
- This concludes our introduction to discrete mathematics, but it should come as no surprise that there is a lot more to learn!